Bricksmith

Agentic AI for commercial real estate — 22 specialist agents that underwrite, close, and manage your deals, already wired into PropAnalyst.

What this product delivers

Bricksmith is a full agentic system — not a prompt pack. 22 LangGraph ReAct agents share one property catalog, one T12/rent-roll store, and one RAG index across leases, zoning memos, environmental reports, property condition reports and title commitments. Agents pass artifacts between each other without the analyst re-keying anything.

It ships with a deterministic synthetic CRE dataset so the system can be demoed, tested and benchmarked end-to-end — no vendor contracts required. Each data adapter is a drop-in replacement, so real CoStar / RCA / broker feeds land without code changes above the data layer.

Demo dataset

Specialist agents	22 across 5 workflow stages
Synthetic properties	40 (MF 15, office 10, industrial 10, retail 5)
Metros covered	8 (Austin, Phoenix, Nashville, Raleigh, Atlanta, Denver, Dallas, Tampa)
Rent-roll line items	~2,600 across the catalog
T12 rows	480 (12 months × 40 properties)
Sales + rent comps	480 (240 sales + 240 rent)
LP CRM contacts	60
Indexed documents (RAG)	237 (leases + zoning + env + PCR + title + market)
Market signal rows	2,640 (24 months × multiple metrics × metros × asset types)

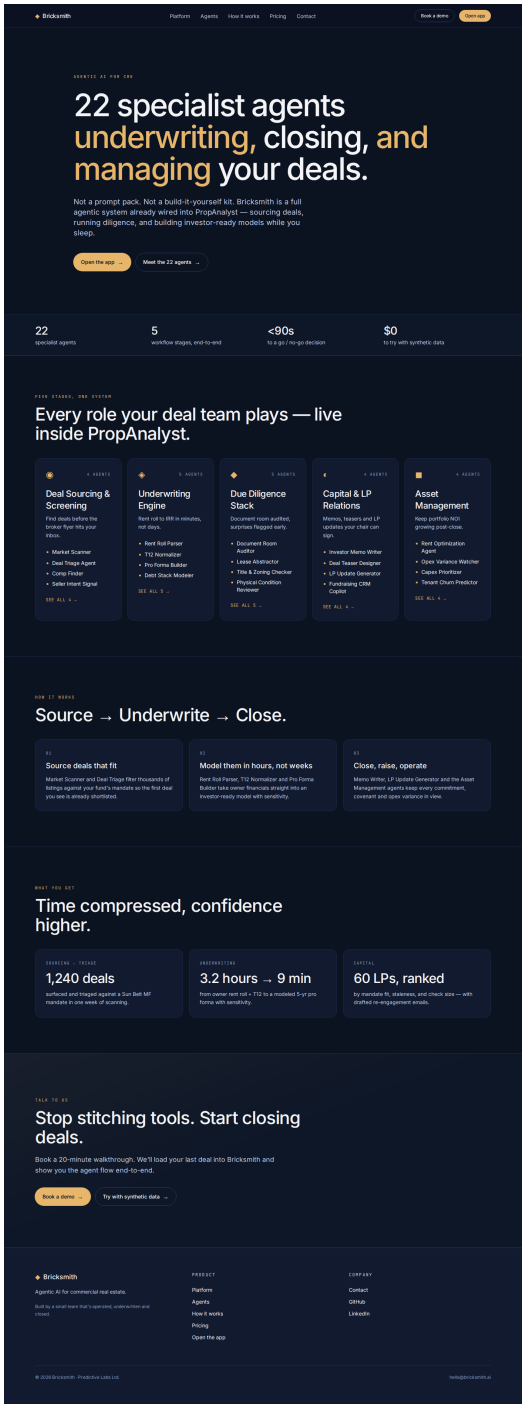
How this guide is organised

The following pages walk through the marketing site and a live chat session, exactly as a new user would see them. All screenshots were captured from the running FastHTML app via Playwright against the synthetic dataset.

Landing page

Agentic AI for CRE — the pitch in 60 seconds.

The home page states the product in one line, names the 22 specialist agents grouped into five workflow stages, and walks a visitor through Source → Underwrite → Close. Sticky navigation lands them on the app with one click.

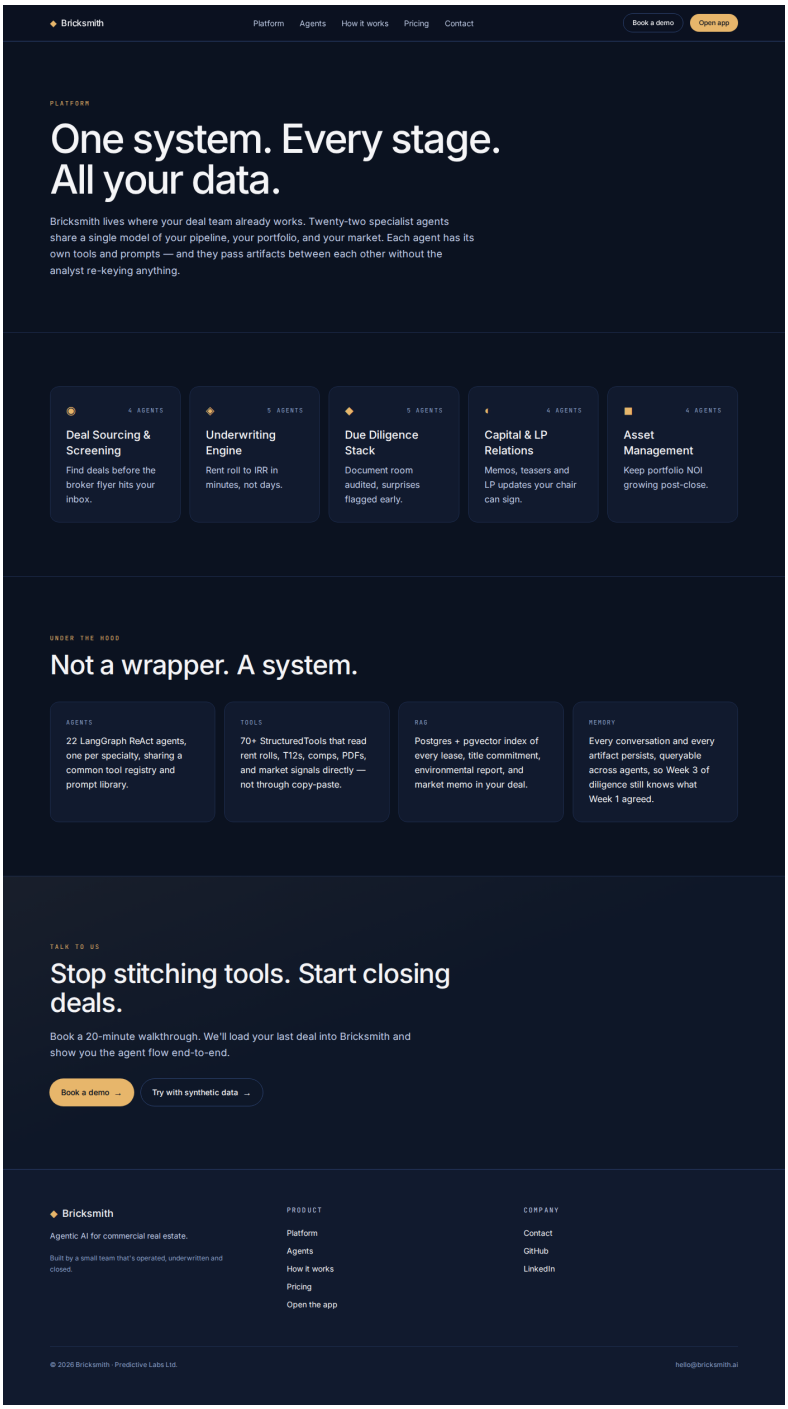


Screenshot: 01-home-full.png

Platform

Not a wrapper — a system.

Platform page breaks down the architecture: agents (LangGraph ReAct, one per specialty), tools (70+ StructuredTools), RAG (Postgres + pgvector across all deal documents), and memory (every conversation and artifact persists across agents).

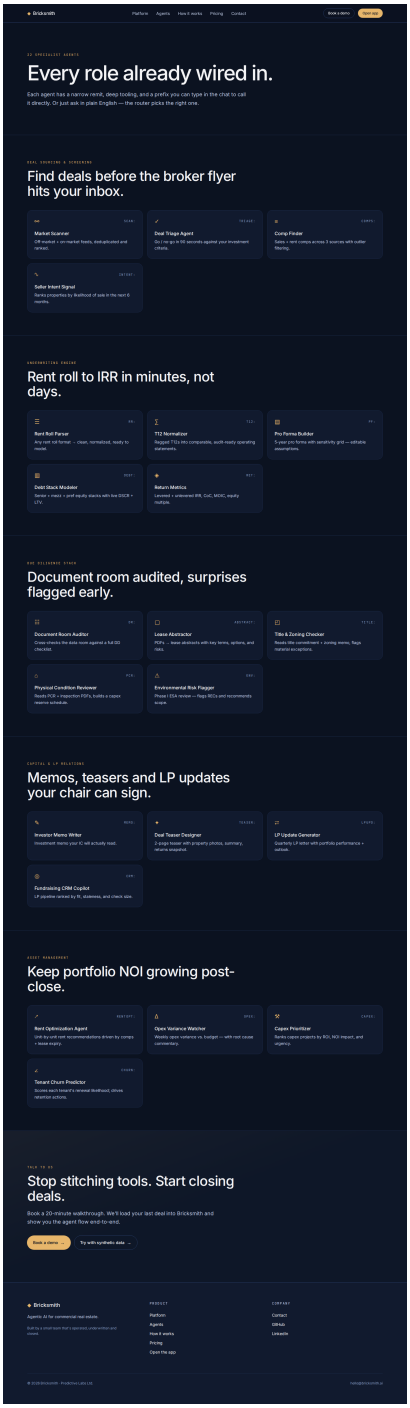


Screenshot: 02-platform-full.png

Agents directory

All 22 agents, grouped.

The agents page renders the registry directly — each agent has a slug, a prefix for direct invocation, and a one-liner. Categories match the deal lifecycle: sourcing, underwriting, diligence, capital / LP, asset management.

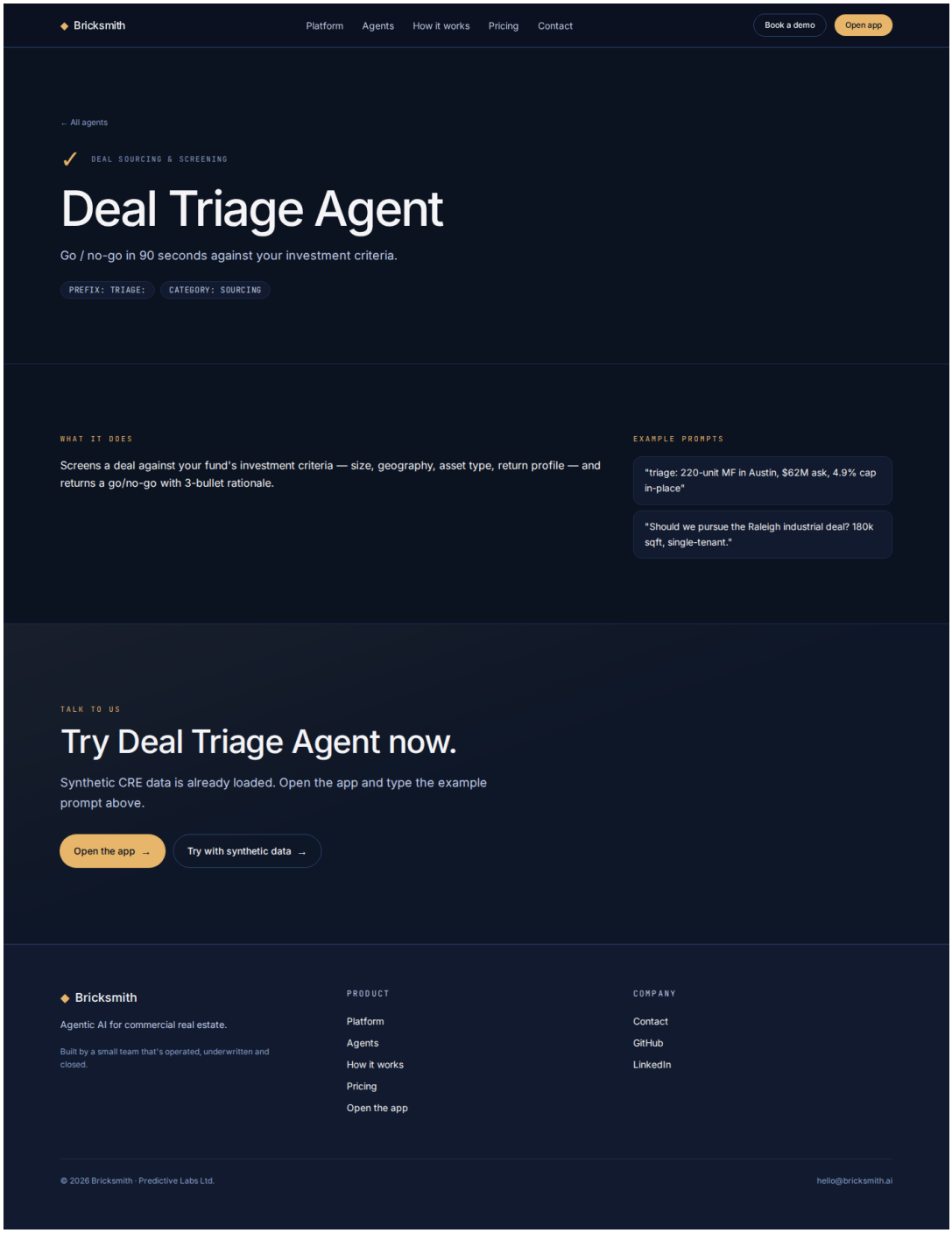


Screenshot: 03-agents-full.png

Agent detail — Deal Triage

90-second go / no-go.

Each agent has its own detail page with the full description, prefix, and concrete example prompts pulled from the registry. One click into the app runs the prompt live against synthetic data.

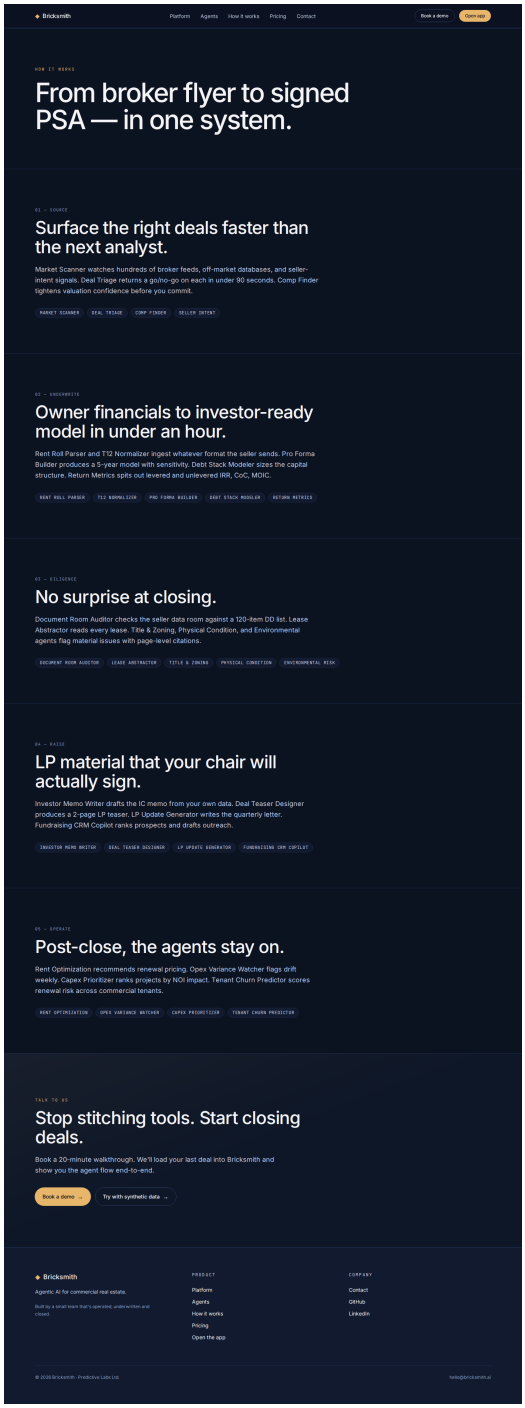


Screenshot: 04-agent-detail-triage.png

How it works

Source → Underwrite → Diligence → Raise → Operate.

The five-stage flow is explained with the participating agents named on each stage, so the reader understands exactly which specialists show up at each point in the lifecycle.

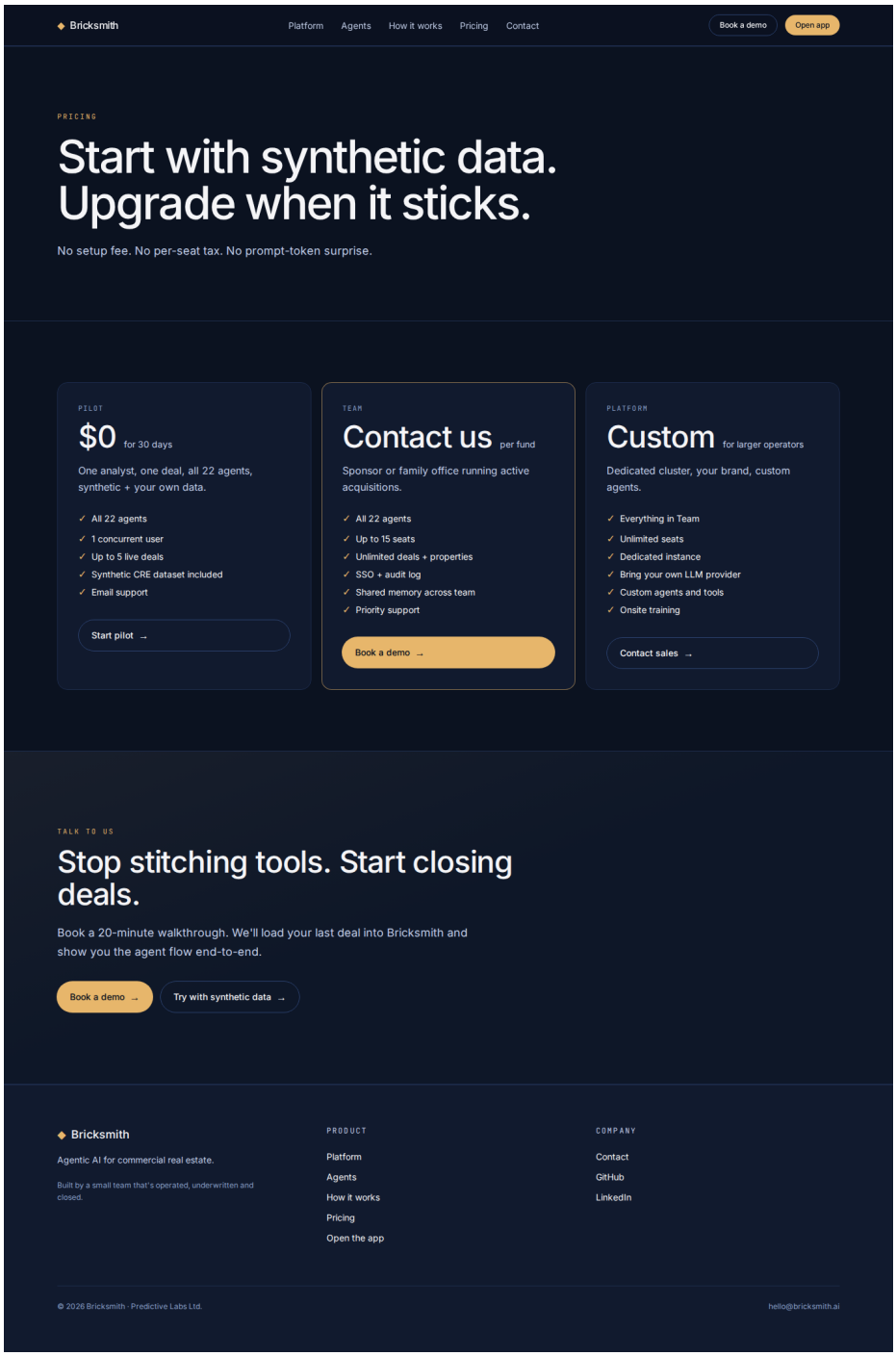


Screenshot: 05-how-it-works-full.png

Pricing

Start with synthetic data. Upgrade when it sticks.

Three tiers — Pilot (free 30 days), Team (per fund), Platform (custom) — scoped around a sponsor's workflow rather than per-seat or per-token metrics.

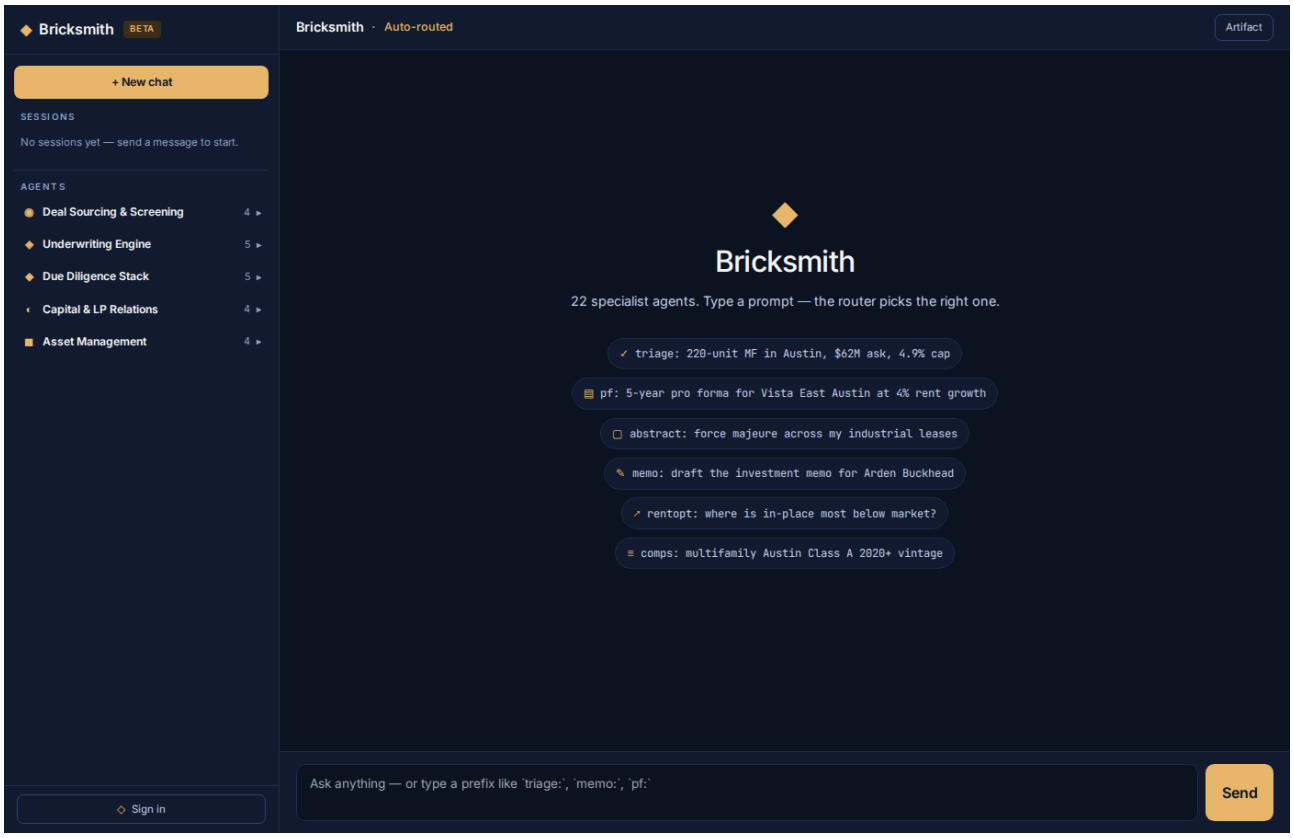


Screenshot: 06-pricing-full.png

Chat — empty state

3-pane product — left agents, centre chat, right artifact.

The chat app mirrors how a deal team actually works: sessions and all 22 agents live on the left, the chat transcript in the centre, and artifacts (property sheets, rent rolls, pro formas, memo previews) on the right. Starter prompts show the prefix syntax.

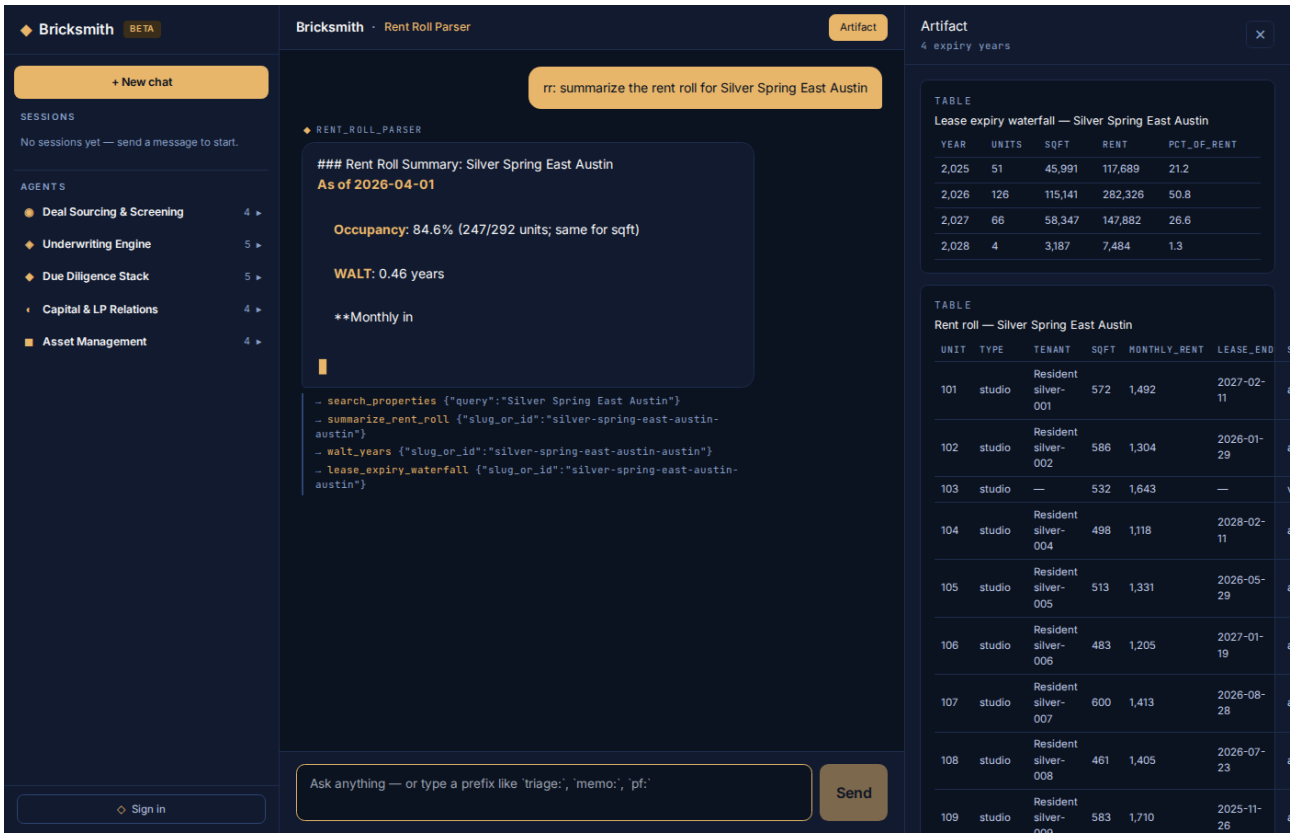


Screenshot: 07-chat-empty.png

Chat — Rent Roll Parser

Tool calls flow into visible artifacts.

`rr:` routes to the Rent Roll Parser. The agent searches the property, loads the current rent roll, computes WALT and the lease-expiry waterfall, and returns a concise commentary with the tables rendered in the right pane.

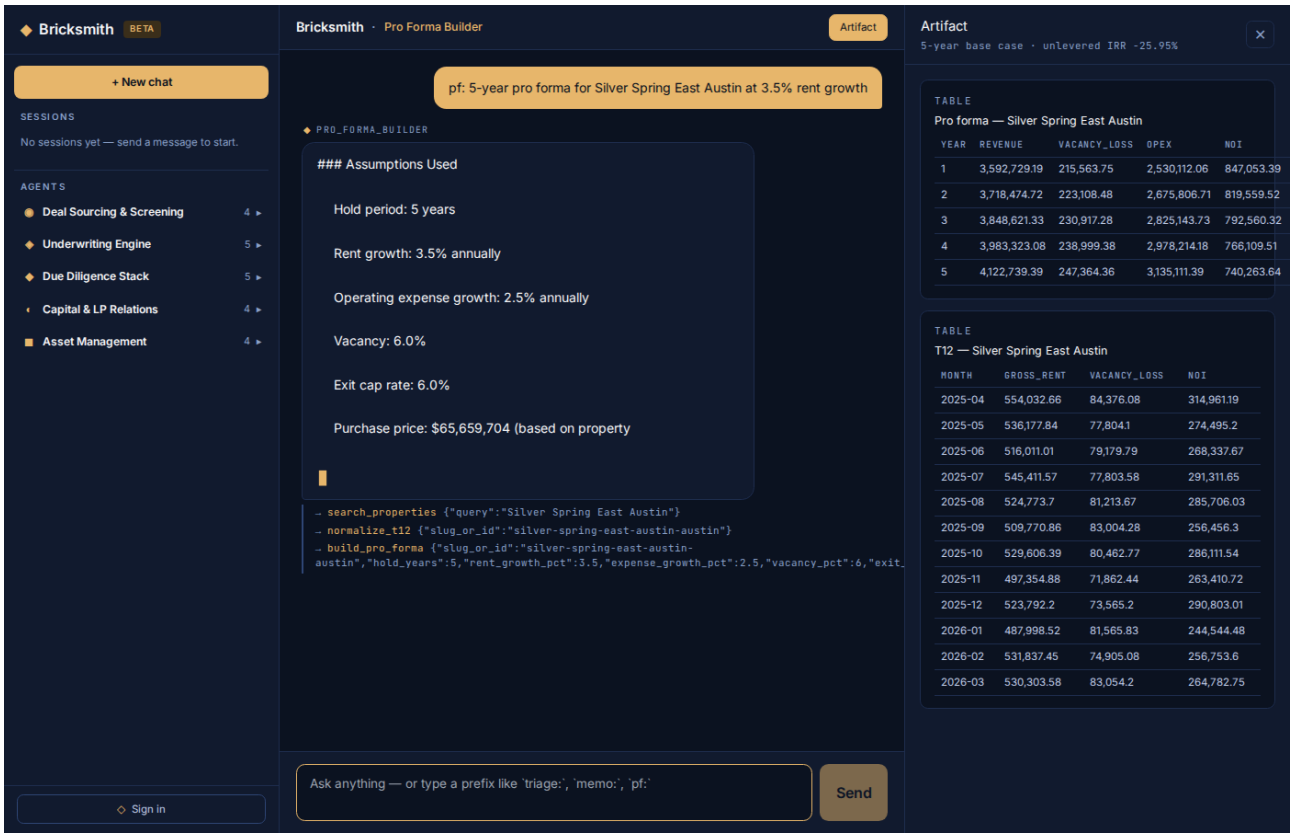


Screenshot: 08-chat-rentroll.png

Chat — Pro Forma Builder

5-year projections with the assumptions stated.

`pf:` routes to the Pro Forma Builder. It normalises the T12, runs a 5-year projection with rent-growth, vacancy, opex-inflation, capex reserve and exit cap — persisting the run in `bricksmith.pro_formas` — and prints the unlevered IRR, CoC and MOIC in-line.

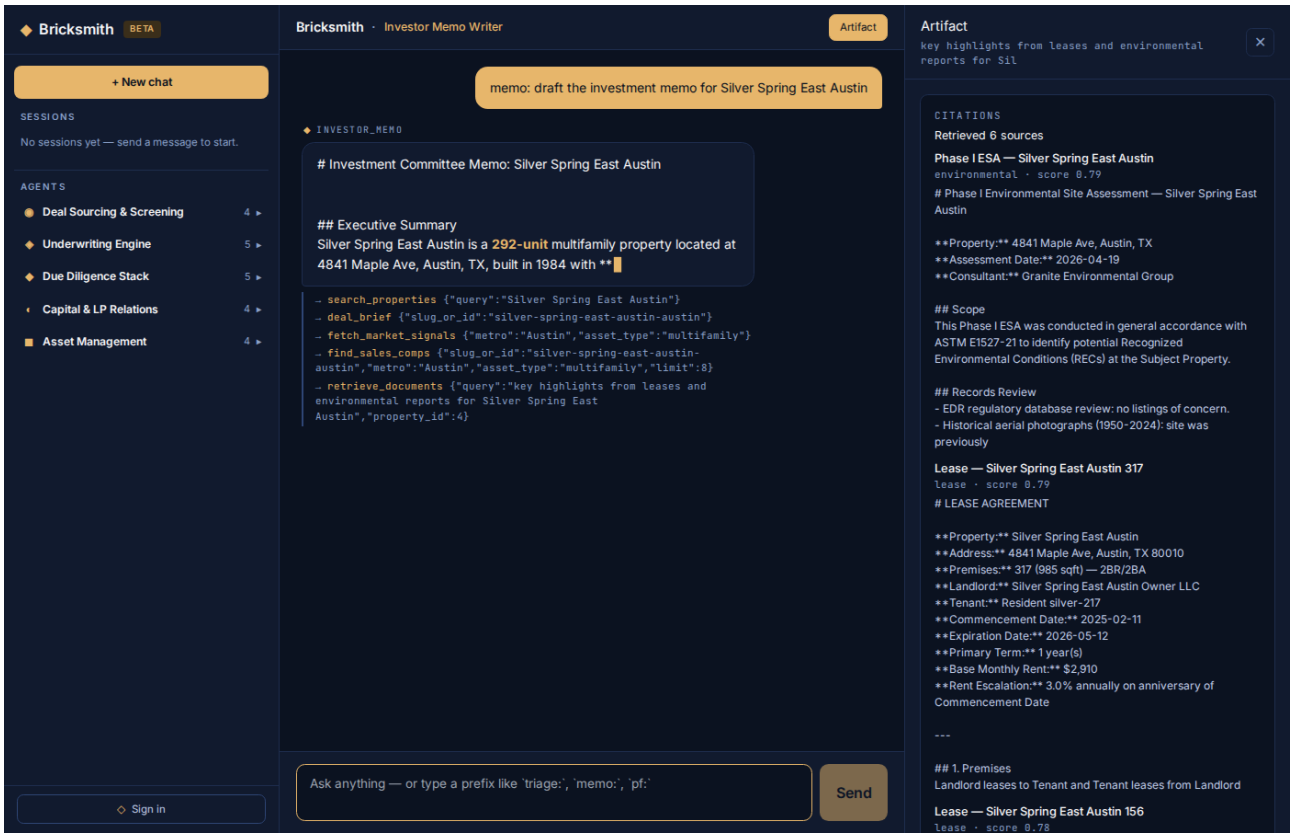


Screenshot: 09-chat-pro-forma.png

Chat — Investor Memo

An IC-ready memo from the deal's own data.

`memo:` routes to the Investor Memo Writer. It pulls the compact deal dossier, supplements with sales comps and market signals, and produces a structured memo — Exec Summary, Strategy, Market, Underwriting, Risks, Recommendation — with every figure cited from tool calls.



Screenshot: 10-chat-memo.png

Installing and running

Requirements: Python 3.12+, PostgreSQL 15+ with pgvector.

```
git clone <repo> bricksmith
cd bricksmith
cp .env.example .env # fill DB_URL, XAI_API_KEY
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m db.migrate
python -m synthetic.generate --seed 42
python main.py
```

Regenerating this guide

Screenshots are captured via Playwright (`scripts/capture_screenshots.py`). The PDF is assembled by `scripts/make_pdf.py`, and the animated GIF by `scripts/make_gif.py`.

All figures in this guide are generated from synthetic data. Do not cite for operational decisions.